A
MAJOR PROJECT REPORT
(SYNOPSIS)
ON

"DSA QUIZ GENERATION USING GEN AI"

This Project Report is submitted to
Rajiv Gandhi Proudyogiki Vishwavidyalaya
in the partial fulfillment for the award of the degree of

Bachelor of Technology
VIII SEM
(Computer Science & Engg.)

Submitted By:

Aradhy Patel (0228CS221043)

Under the guidance of:

Prof. Nitin Shukla
Head of Department
Computer Science
SHRI RAM GROUP OF INSTITUTIONS (SRGI)

SHRI RAM GROUP OF INSTITUTIONS (SRGI), JABALPUR
2026

SHRI RAM GROUP OF INSTITUTIONS (SRGI), JABALPUR

Approved by AICTE New Delhi & Govt. of M.P.
(Affiliated to R.G.P.V. - University of Technology of Madhya Pradesh)

CERTIFICATE

This is to certify that,

Aradhy Patel (0228CS221043)

Students of 8th Semester, Computer Science & Engineering, **SHRI RAM GROUP OF INSTITUTIONS (SRGI)**, Jabalpur have submitted their major project synopsis entitled "**DSA QUIZ GENERATION USING GEN AI**" for the completion of 8th semester examination under the requirement for the degree of Bachelor of Technology as per **R.G.P.V., Bhopal (M.P.).**

This is an original work of these students and is based on their own innovative idea. We, hereby certify this project to be an authentic work by these young minds.

Prof. Nitin Shukla
Head- Computer Science Dept.
S.R.G.I.

CERTIFICATE OF APPROVAL

The foregoing major project title **" DSA QUIZ GENERATION USING GEN AI"** is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite to the degree for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner) **(External Examiner)**

DECLARATION

Here we declare that the project entitled which is being submitted in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science "***RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA, BHOPAL [M.P]***" is the authentic record of my own work done under the guidance of **Prof. Nitin Shukla**[HOD - Computer Science Dept].

The Matter reported in this project has not been submitted earlier for the award of degree.

PLACE: JABALPUR Aradhy Patel(0228CS221043)

DATE:

SHRI RAM GROUP OF INSTITUTIONS (SRGI), JABALPUR

Approved by AICTE New Delhi & Govt. of M.P.
(Affiliated to R.G.P.V. - University of Technology of Madhya Pradesh)

FORWARD

I hereby forward project title **"DSA QUIZ GENERATION USING GEN AI"** *which is being submitted by*

Aradhy Patel(0228CS221043)

I**n** partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Computer Science "***Rajiv Gandhi Proudyogiki Vishwavidyalaya Bhopal [M.P]"***

PLACE: **JABALPUR**
DATE:

Prof. Nitin Shukla

HOD
Computer Science & Engg. .Department
SHRI RAM GROUP OF INSTITUTIONS
(SRGI) JABALPUR [M.P]

ACKNOWLEDGEMENTS

We would like to extend our gratitude to all those who gave us the possibility to complete this project. We would like to thank the Department of computer science of SHRI RAM GROUP OF INSTITUTIONS (SRGI) for giving us permission to commence this project in first instance, to do the necessary research and to use departmental accessories.

We are deeply indebted to our supervisor Prof. Nitin Shukla [HOD -Computer Science Dept] whose help, simulating suggestions and encouragement helped us in all the times in completing this project.

We are bound to our honorable head of the department Prof. Nitin Shukla his simulating support. We have further more to thank the Faculty of our college who helped us in providing the necessary data.

Our former colleagues in the department also supported us in our project. We want to thank them for all their help, support, interest and valuable hints.

PLACE: **JABALPUR** **STUDENT NAME: Aradhy Patel**
DATE:

INDEX

Chapter 1	**INTRODUCTION**	**1–4**
	1.1 Introduction 1.2 Problem Statement 1.3 Project Objectives & Goals 1.4 Scope and Limitations 1.5 Report Outline	1 2 2 3 4
Chapter 2	**LITERATURE SURVEY**	**5-6**
	2.1 Survey and Analysis of Existing Solutions 2.2 Identification of Research Gap	5 6
Chapter 3	**REQUIREMENT ANALYSIS AND SPECIFICATION**	**7-9**
	3.1 Functional Requirements 3.2 Non-Functional Requirements 3.3 Software and Hardware Requirements	7 7 8
Chapter 4	**SYSTEM DESIGN AND ARCHITECTURE**	**10–17**
	4.1 System Architecture Diagram 4.2 System Module Design 4.3 Model Design (Flowcharts, Pseudocode) 4.4 DFD Diagrams 4.5 Use Case Diagrams	10 11 13 15 17
Chapter 5	**IMPLEMENTATION, TESTING, AND MAINTENANCE**	**18–20**
	5.1 Introduction to Languages, IDE's	18
	5.2 Tools and Technologies used for Implementation	18
	5.3 Coding standards of Language used Project Scheduling using various tools such as PERT, GANTT charts, Open PROJ etc.	19
	5.4 Testing Techniques and Test Plans	20
Chapter 6	**EXPECTED RESULTS AND DISCUSSION**	**21–23**
	6.1 Expected Results and Discussion 6.2 Expected Performance Results	21 **21**
Chapter 7	**CONCLUSION**	**24**
	7.1 Conclusion	24
	REFERNECES	**25**
APPENDICES	Appendix A: Source Code (GitHub Repository Link and Snapshot) Appendix B: Comprehensive Project Timeline (Gantt Chart) Appendix C: Team Member Contributions Appendix D: Dataset Details and Sources Appendix E: Relevant Published Papers	26 26 27 27 28

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The rapid evolution of Generative Artificial Intelligence (GenAI) has fundamentally restructured modern educational technology and automated assessment systems. In the current academic landscape, the demand for technically rigorous evaluation material, particularly in specialized domains like Data Structures and Algorithms (DSA), is at an all-time high. Traditional quiz creation often relies on static question banks or intensive manual labor, a process that is time-consuming, prone to error, and results in repetitive content that fails to effectively challenge students.

In response, this project introduces "DSA Quiz Generation Using GenAI," a stateless "Question Factory" developed at the Shri Ram Group of Institutions, Jabalpur. Utilizing the Llama-3.1-8b-instant model orchestrated via LangChain and Groq's high-speed inference engine, the system enables real-time generation of unique coding challenges. By leveraging a Next.js frontend and Python backend, educators can generate descriptive, machine-readable questions on-the-fly without the overhead of persistent database management.

The core implementation involves transforming user requirements into strictly validated JSON outputs through temperature-controlled inference and regex-based post-processing. This ensures that every quiz is conceptually distinct, structurally sound, and ready for immediate deployment, demonstrating how GenAI can practically enhance programming education and reduce manual workloads.

1.2 PROBLEM STATEMENT

In modern academic and professional environments, generating high-quality programming and Data Structures and Algorithms (DSA) assessments is a stagnant, time-intensive, and mentally demanding task. Instructors frequently struggle to produce a high volume of unique, conceptually accurate coding problems, leading to repetitive content and inconsistent difficulty levels. As the demand for automated assessment grows, manual curation has become an unsustainable bottleneck.

A significant technical challenge lies in the requirement for structured, machine-readable formats. Most manually produced questions lack the precise formatting—such as specific constraints and sample test cases—necessary for seamless integration into automated testing platforms. Even when utilizing Generative AI, raw outputs are often unreliable due to hallucinations, formatting errors, and a lack of structural validation.

Consequently, there is a critical need for a robust, real-time, GenAI-driven system capable of generating high-quality DSA questions in a strictly validated JSON format. This project addresses the absence of a scalable "Question Factory" that ensures every output is instructionally relevant, non-repetitive, and unique. By implementing a stateless architecture with an automated validation layer, the system eliminates the overhead of long-term database management while providing reliable, diverse, and structured assessment material for digital learning environments.

1.3 PROJECT OBJECTIVES & GOALS

The primary objective of this project is to develop a real-time, automated pipeline for generating high-quality Data Structures and Algorithms (DSA) quiz questions using generative AI. By transitioning from manual curation to an on-demand system, the project drastically reduces the human effort required to produce diverse assessment material. A fundamental goal is implementing a prompt-driven

architecture that translates user-defined parameters—such as difficulty and topic—into highly structured system instructions.

To ensure structural integrity, the system produces detailed problem statements with narratives, examples, and constraints, strictly adhering to a validated JSON schema. By integrating regex-based parsing and automated retry logic, the project overcomes common AI consistency issues to guarantee error-free, machine-readable responses. This ensures that every output is technically precise and ready for integration into digital platforms.

Furthermore, the project mandates mutual exclusivity across topics to ensure conceptual richness and prevent redundancy. From an implementation standpoint, the goal is to provide a seamless, stateless experience through a responsive React.js interface and a Python-based inference engine. Ultimately, this project bridges the gap between raw AI capabilities and structured academic workflows, demonstrating a practical and scalable application of generative AI in programming education.

1.4 SCOPE AND LIMITATIONS

The project scope encompasses an AI-powered, stateless quiz generation system designed to produce structured DSA questions based on user-defined topics and difficulty levels. Powered by the Llama-3.1-8b-instant model, the system generates descriptive problem statements, technical constraints, and examples. A core component of this scope is the development of a validation pipeline using regex and retry logic to ensure outputs strictly adhere to a machine-readable JSON schema, deployed as a functional proof-of-concept on Render.

However, certain limitations define the system's boundaries. The quality of questions remains dependent on the AI's internal logic, occasionally requiring oversight for complex cases. Due to the stochastic nature of LLMs, 100% flawlessness cannot be guaranteed. Additionally, the system is restricted to text-based questions, lacking support for graphical visualizations like tree diagrams.

Finally, the absence of a persistent database means generated data is not stored after a session, emphasizing its function as a real-time tool.

1.5 REPORT OUTLINE

This project report is systematically organized into several chapters that collectively describe the conceptual foundations, development methodology, and technical execution of the "DSA Quiz Generation Using GenAI" system. Chapter 1 serves as the introductory foundation, establishing the background, problem statement, and technical objectives while defining the stateless scope of the project. Chapter 2 provides a comprehensive literature survey, evaluating existing static and semi-automated solutions to identify the research gap that necessitates a real-time, AI-driven JSON generator. Chapter 3 details the requirement analysis, specifying the functional parameters and the hardware-software stack, such as the Groq API and LangChain.

Chapter 4 presents the system design and architecture, illustrating the modular pipeline from the Next.js frontend to the Python backend orchestrator. Chapter 5 is a new addition focusing on implementation and testing, providing deep insights into the regex-based parsing logic, coding standards, and rigorous test plans used to ensure model reliability. Chapter 6 discusses the expected performance outcomes and the accuracy of the Llama-3.1-8b model under various difficulty levels. Finally, Chapter 7 concludes the report by summarizing the project's contributions to educational technology and suggesting future enhancements for adaptive learning.

CHAPTER 2
LITERATURE SURVEY

2.1 SURVEY AND ANALYSIS OF EXISTING SOLUTIONS

The evolution of digital learning platforms and coding education has given rise to multiple systems that support automated assessments and algorithmic problem creation. However, the analysis of the current landscape reveals that most existing solutions fall into two broad categories: static question banks curated manually by experts, and partially automated systems that depend significantly on human involvement. Traditional programming platforms such as GeeksforGeeks, LeetCode, and HackerRank offer extensive collections of coding questions, but these repositories are inherently static. They involve costly and time-consuming manual content development by subject-matter experts, which limits their scalability and prevents the real-time generation of custom quizzes tailored to specific, immediate learning outcomes.

In recent years, semi-automated systems have attempted to bridge this gap using templates or procedural content generation. While these algorithmic approaches work effectively for subjects like mathematics, they often fail in the programming domain, where question design requires high levels of creativity, descriptive narrative-based problem statements, and complex contextual constraints. Template-driven generators frequently produce repetitive structures, which compromises the originality of the questions and reduces the overall effectiveness of the assessment.

With the rise of Large Language Models (LLMs) like GPT and Llama, there has been a surge in utilizing AI for educational content creation. Research confirms that these models can generate coding questions with notable fluency. However, raw outputs from these models often suffer from structural inconsistencies, "hallucinations," and a lack of strict adherence to the machine-readable schemas required by automated evaluation systems. Most public AI tools do not provide the deterministic validation mechanisms or retry pipelines necessary to ensure that

generated content meets the rigorous structural requirements of programmatic workflows.

2.2 IDENTIFICATION OF RESEARCH GAP

Despite the significant progress in AI-assisted education, a substantial research gap exists in the domain of automated programming quiz generation, specifically regarding systems that provide validated, structured, and topic-specific coding problems without the overhead of persistent storage. Current solutions fall short in generating questions that are both pedagogically sound and technically formatted for seamless integration with modern digital platforms. One major gap lies in the inability of existing AI tools to consistently produce content in strict machine-readable formats such as JSON. Large language models, while capable of generating rich text, often struggle to maintain structural accuracy, frequently embedding irrelevant explanations, incorrect nesting, or inconsistent formatting that breaks automated pipelines.

Another research gap emerges in the lack of systems that ensure diversity and mutual exclusivity across generated questions. Without properly enforced prompt engineering, AI-generated questions can become redundant or conceptually overlapping. Existing tools rarely incorporate a "stateless" validation layer that can detect repetition in real-time or mandate that questions be inspired by descriptive, real-world scenarios.

Furthermore, there is a deficiency in error-handling frameworks that can automatically trigger a retry mechanism upon detecting invalid output or mismatched question counts. Most existing systems do not cater specifically to the deep structural needs of programming education, which require detailed constraints and input-output formatting. This project bridges these gaps by introducing an integrated, stateless solution that combines the creative power of Llama-3.1-8b with strict regex-based validation and recursive retry logic, ensuring high-quality, unique, and machine-ready assessments.

CHAPTER 3

REQUIREMENT ANALYSIS AND SPECIFICATION

3.1FUNCTIONAL REQUIREMENTS

The functional requirements of the "DSA Quiz Generation Using GenAI" system define the core operations that the software must perform to meet its intended purpose of generating dynamic, high-quality programming quiz questions. At the center of the system is the functionality that allows users to specify key inputs such as the number of questions, specific difficulty levels, and various DSA-related topics—including arrays, linked lists, trees, graphs, recursion, and dynamic programming. Once these parameters are submitted via the Next.js interface, the system must dynamically generate detailed coding problems using the Llama-3.1-8b-instant model.

A critical functional requirement is the strict adherence to a predefined JSON schema, ensuring that the output includes a descriptive problem statement, difficulty level, sample input-output examples, and constraints. To handle the stochastic nature of AI, the system must implement a validation and parsing mechanism that utilizes Regular Expressions to extract the correct JSON segment and verify its structural integrity. Furthermore, if the generated output fails validation or the question count is incorrect, the system must automatically trigger a retry-based correction mechanism. Finally, the system must ensure that generated questions are unique, conceptually distinct, and inclusive of all requested difficulty levels.

3.2 NON-FUNCTIONAL REQUIREMENTS

In addition to core functional capabilities, the non-functional requirements define the standards for performance, reliability, and usability that ensure the "DSA Quiz Generation Using GenAI" system operates effectively in a real-world academic environment. The most critical non-functional requirement is performance

efficiency; the system is expected to generate complex DSA test cases with minimal latency, ensuring a responsive user experience by optimizing the communication between the Next.js frontend and the Groq API.

Reliability is equally paramount, as the system must consistently produce valid, machine-readable JSON even when the underlying LLM provides conversational filler. This is achieved through the validation layer and a robust retry mechanism that filters out malformed data. Scalability is addressed through the stateless architecture, allowing the system to handle concurrent generation requests without the performance degradation typically associated with heavy database transactions. Furthermore, maintainability is ensured by adhering to modular Python programming practices and PEP 8 standards, making it easy to swap the Llama-3.1-8b model for newer versions in the future. Finally, security is maintained by using environment files to protect sensitive API keys, ensuring unauthorized access to the inference engine is prevented.

3.3 SOFTWARE AND HARDWARE REQUIREMENTS

The development and operation of the "DSA Quiz Generation Using GenAI" system rely on a specialized stack of software frameworks and hardware resources designed to support high-speed AI inference and a responsive user interface. On the software side, the application is built using Python as the primary backend language, chosen for its robust ecosystem of machine learning libraries and JSON manipulation utilities. The project utilizes the LangChain framework to interface with the Llama-3.1-8b-instant model through Groq’s inference infrastructure. The frontend is developed using Next.js, ensuring a component-based and reactive user dashboard. Additional libraries such as python-dotenv are employed for secure API key management, while regex and json modules handle the critical task of data parsing and validation.

On the hardware side, the system benefits from a hybrid cloud architecture, which keeps local requirements modest. Since the heavy computational tasks associated with LLM inference are performed on Groq’s remote servers, a standard

development machine with at least 8 GB of RAM and a multi-core processor is sufficient for local development and testing. However, stable internet connectivity is a mandatory hardware requirement to maintain the API link between the backend and the cloud-based AI engine. The final application is hosted on Render, utilizing cloud-based server resources to ensure high uptime and accessibility.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 SYSTEM ARCHITECTURE DIAGRAM (Content)

The system architecture for the "DSA Quiz Generation Using GenAI" platform is designed as a streamlined, stateless pipeline that facilitates high-speed communication between the user and the Llama-3.1-8b-instant model. This architecture is specifically engineered to operate without a primary database, prioritizing real-time "on-the-fly" generation over long-term data persistence. The design is divided into three distinct layers: the Frontend Layer, the Orchestration Layer, and the Inference Layer.

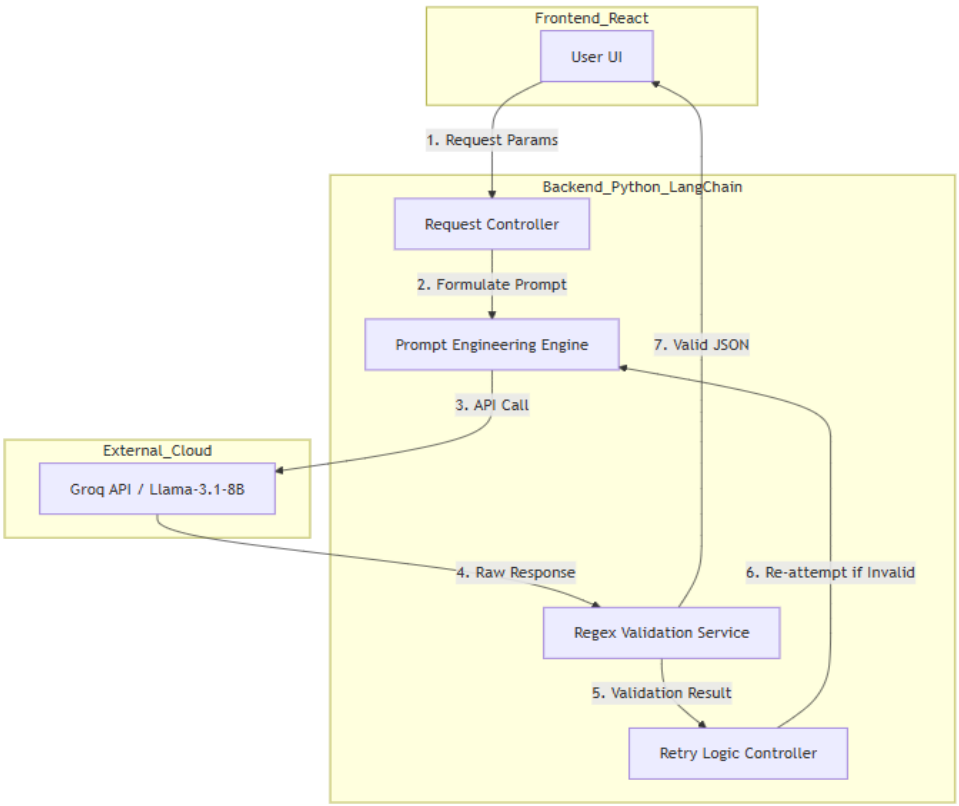

Figure – System Architecture Diagram

The process begins at the Frontend Layer, where a Next.js interface collects structured inputs from the user, including specific DSA topics, question count, and difficulty distribution. This data is transmitted via secure API calls to the Orchestration Layer, a Python-based backend powered by LangChain. Here, the system performs Prompt Engineering, converting user requirements into a strict system prompt that mandates "mutually exclusive" and descriptive problem statements.

The Inference Layer involves the Groq API, which executes the AI model and returns a raw string. Before reaching the user, the response passes through a Regex Validation Service. This service extracts the valid JSON block and, if the structural requirements or question counts are not met, communicates with the Retry Logic Controller to trigger a re-generation attempt. This stateless flow ensures that the system remains lightweight, responsive, and compatible with modern web environments.

4.2 SYSTEM MODULE DESIGN

Since the system operates on a stateless architecture, the internal design is organized into distinct logical modules that handle the lifecycle of a single request without the need for persistent local storage. This modular approach ensures a clear separation of concerns, facilitating easier debugging and future updates to the generative engine.

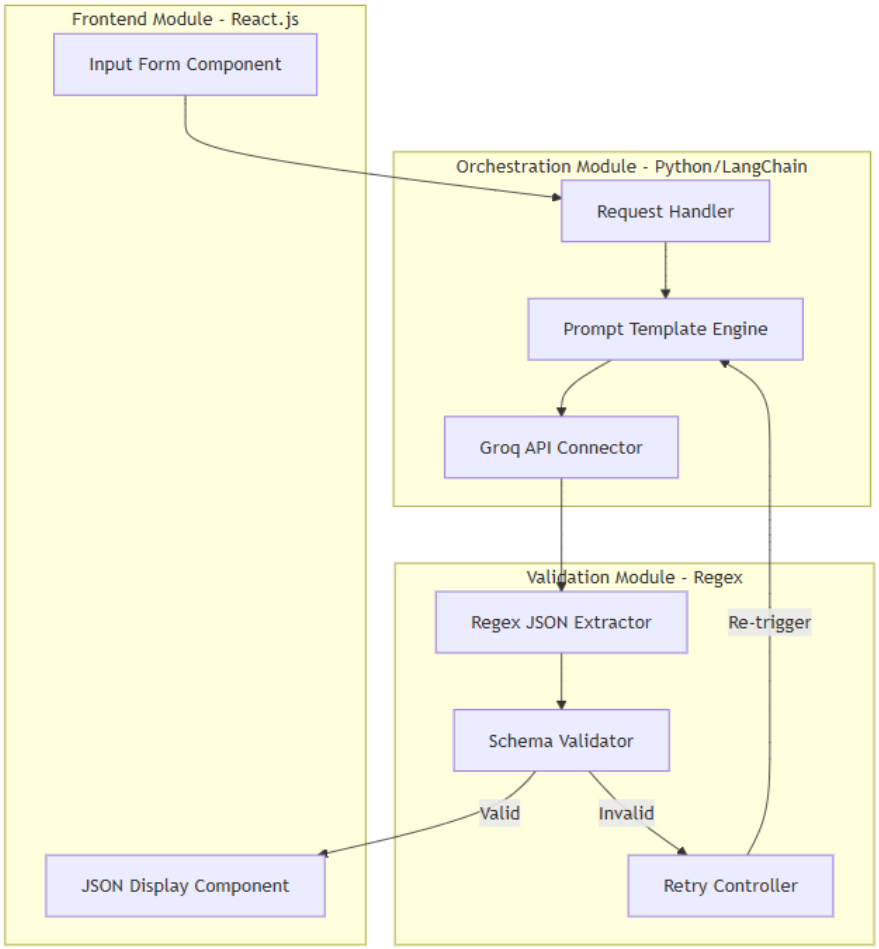

Figure – System Module Design Diagram

The User Interface Module, developed with Next.js, serves as the entry point where user parameters are collected and validated on the client side. Once submitted, the AI Orchestration Module—built using Python and LangChain—manages the complexity of prompt engineering and API communication with the Groq cloud layer. This module is responsible for translating user intent into high-fidelity instructions that the Llama-3.1-8B model can execute. Finally, the Validation and Parsing Module acts as a technical gatekeeper; it utilizes specialized Regex logic to extract raw JSON from the AI's conversational response and monitors the output for structural compliance. If the criteria for question count or format are not met, this module triggers the Retry Logic, ensuring that only high-quality, valid data is delivered back to the UI.

4.3 MODEL DESIGN (FLOWCHARTS, PSEUDOCODE)

The model design for the "DSA Quiz Generation Using GenAI" system provides a systematic blueprint of the logic governing prompt orchestration, AI invocation, and structured output verification. The process begins when the backend receives a user request containing specified topics, question counts, and difficulty distributions. The system immediately validates these parameters before constructing a specialized prompt that conforms to a strict JSON schema. This prompt is engineered with high-level constraints to enforce rules such as conceptual uniqueness and the inclusion of descriptive, real-world inspired problem statements.

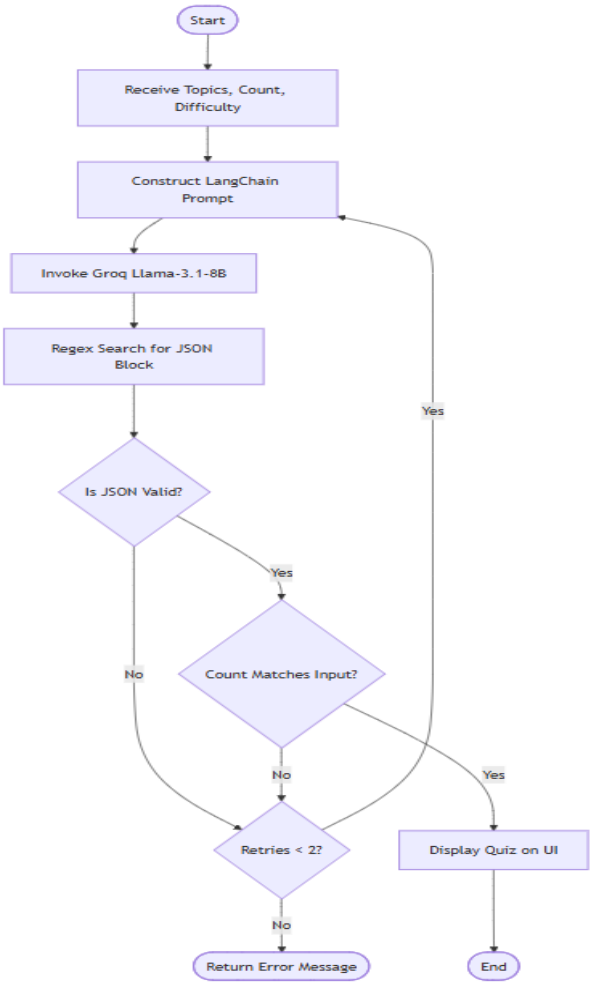

Figure – System Flow Chart

Once the prompt is dispatched to the Llama-3.1-8b-instant model, the system enters a critical verification phase. The backend retrieves the raw string response and initiates a Regex-based search to isolate the JSON block from any surrounding conversational text. The model design incorporates a recursive Retry Mechanism; if the extracted JSON is malformed or if the number of generated questions does not match the user's initial requirement, the system automatically triggers a re-generation attempt. This ensures that the operational logic remains robust, delivering only high-fidelity, machine-readable data to the final user interface while maintaining a stateless and efficient execution flow.

Pseudocode for Model Logic

This pseudocode represents the operational algorithm of your GenAINew.py script. You can include this as a code block or a formatted figure in your report.

Python

```
FUNCTION generate_quiz(num_questions, topics, difficulties):
    INITIALIZE retries = 2
    CONSTRUCT prompt_template WITH user_inputs
    FOR attempt FROM 1 TO retries:
        response = INVOKE_MODEL(prompt_template)
        parsed_data = REGEX_EXTRACT_JSON(response.content)
        IF parsed_data IS NOT NULL:
            IF length(parsed_data) EQUALS num_questions:
                RETURN parsed_data
            ELSE:
                LOG "Question count mismatch"
        ELSE:
            LOG "JSON structural error"
```

RETURN ERROR "Generation failed after retries"

4.4 DATA FLOW DIAGRAMS (DFD)

The Data Flow Diagrams (DFD) provide a structural visualization of how information moves through the "DSA Quiz Generation Using GenAI" system, moving from initial user input to the final delivery of structured questions. At the highest level, the Level 0 DFD (Context Diagram) represents the entire system as a single process, showing the primary data exchange between the User and the Groq Cloud AI. The user provides parameters such as topics and difficulty, while the system interacts with the external AI model to retrieve raw content.

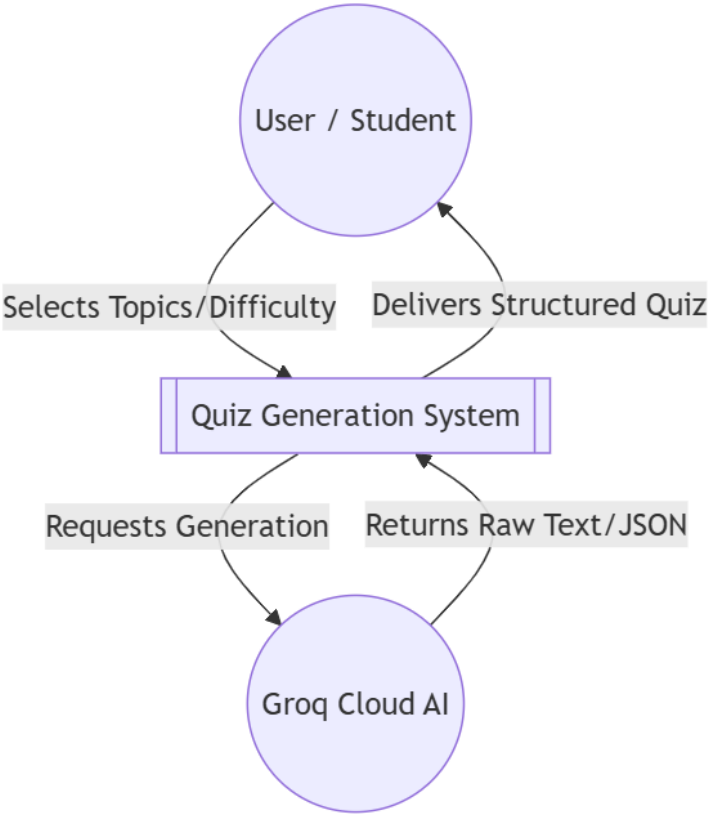

Figure – DFD level 0

The Level 1 DFD decomposes this process into internal subprocesses to illustrate the detailed data lifecycle. It begins with the Input Validation subprocess, which ensures parameters are consistent, followed by Prompt Construction to generate

the model instructions. The data then flows through the AI Interaction subprocess to capture the response and into the Validation Subprocess, where Regex-based parsing and count verification occur. If validation fails, the Retry Subprocess initiates a loop to re-generate the data. Finally, the validated data proceeds to the User Display subprocess, completing the cycle without the need for intermediate database storage.

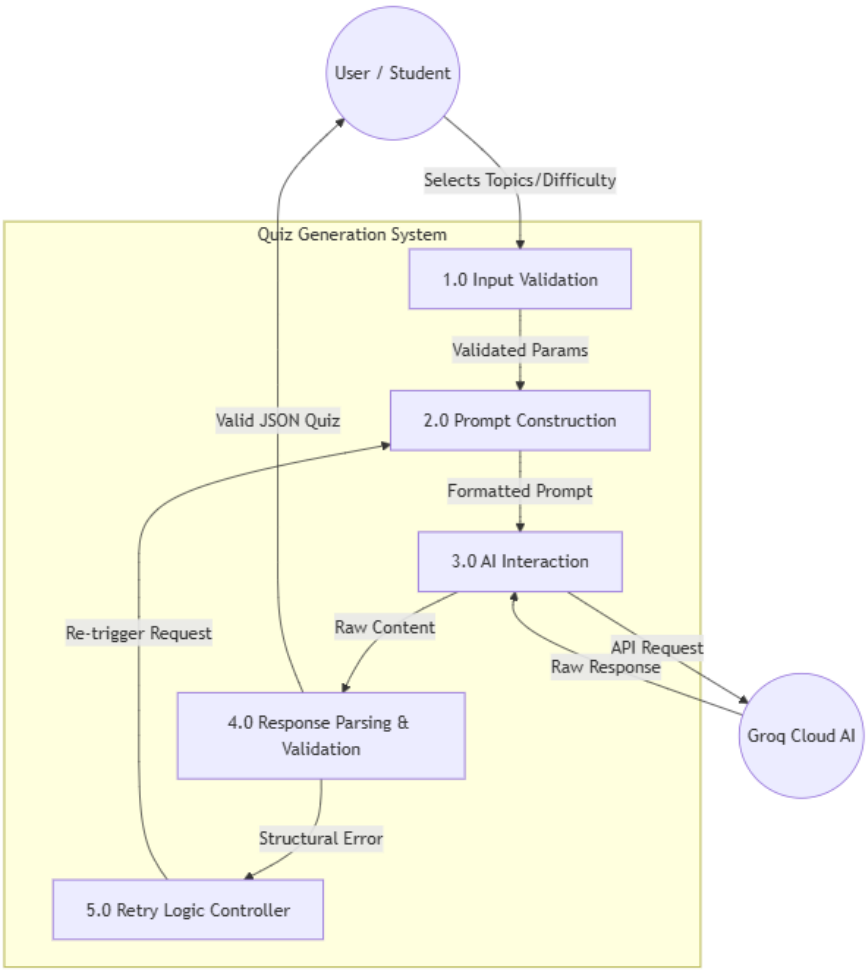

Figure – DFD level 1

4.5 USE CASE DIAGRAMS

The use case diagrams represent the critical interactions between the primary actor and the system's core functionalities within a stateless environment. The main actor in this system is the User (Student, Instructor, or Developer), who interacts with the platform to fulfill specific educational needs. Unlike traditional systems that include "Review History" or "Profile Management" use cases, this model focuses exclusively on the real-time generation and consumption of AI-driven content. The primary use case is "Generate AI Quiz," where the user provides parameters and triggers the backend pipeline to receive structured JSON questions.

A detailed look at the use case framework reveals that the "Generate AI Quiz" process includes two vital automated sub-use cases: "Format Validation" and "Retry Logic Execution". These are system-initiated actions that ensure the structural integrity of the output without requiring manual user intervention. Additionally, the "View Structured Questions" use case allows the user to interact with the final output directly on the React interface. By mapping these interactions, the diagram clarifies the system boundaries and emphasizes a user-centric design that prioritizes reliability and immediate instructional value over long-term data storage.

CHAPTER 5
IMPLEMENTATION, TESTING, AND MAINTENANCE

5.1 INTRODUCTION TO LANGUAGES, IDEs

The implementation of the "DSA Quiz Generation Using GenAI" project utilizes a specialized multi-language stack designed to handle complex AI inference tasks while maintaining a lightweight and highly responsive user interface. The backend logic is developed using Python 3.10+, selected for its unparalleled ecosystem of libraries dedicated to Large Language Model (LLM) orchestration and structured data manipulation. For the client-facing layer, JavaScript paired with the Next.js framework is employed to create a component-based dashboard. This ensures that the frontend can handle asynchronous API responses efficiently, providing users with a seamless experience where questions appear in real-time without requiring full page reloads.

The primary Integrated Development Environment (IDE) used throughout the development lifecycle was Visual Studio Code (VS Code). It was chosen for its robust support for Python virtual environments and its integrated terminal, which allowed for the simultaneous management of the React development server and the FastAPI/Flask backend logic. To ensure code reliability and security, the environment was configured using python-dotenv to isolate sensitive API keys and Virtualenv to prevent dependency conflicts between modular scripts. This combination of modern languages and a streamlined IDE setup facilitated the rapid prototyping and deployment of the stateless "Question Factory" architecture.

5.2 TOOLS AND TECHNOLOGIES USED FOR IMPLEMENTATION

The technical foundation of the "DSA Quiz Generation Using GenAI" system relies on a curated suite of high-performance tools that facilitate seamless AI orchestration and rapid data processing. At the core of the inference engine is the

Llama-3.1-8b-instant model, a state-of-the-art large language model optimized for low-latency text generation. This model is accessed through the Groq Cloud API, which provides an ultra-fast inference infrastructure that is essential for maintaining a responsive user experience in a real-time environment. By offloading heavy computational tasks to Groq's specialized hardware, the system achieves near-instantaneous question generation without requiring local GPU resources.

The orchestration layer is powered by LangChain, which manages the complex interactions between the frontend parameters and the AI model. LangChain facilitates the use of ChatPromptTemplates, allowing for the dynamic insertion of user-selected topics and difficulty levels into a rigid system prompt. To bridge the gap between AI-generated text and a functional UI, the system utilizes Python's regex module for structural validation and Next.js for the frontend dashboard. Finally, the entire application is hosted on Render, a unified cloud platform that provides the necessary scalability and accessibility for public testing and deployment.

5.3 CODING STANDARDS OF LANGUAGE USED & PROJECT SCHEDULING

The development of the "DSA Quiz Generation Using GenAI" system adheres to rigorous coding standards to ensure long-term maintainability, readability, and security within a stateless architecture. For the backend, the project strictly follows PEP 8 (Python Enhancement Proposal 8) guidelines, which mandate standardized naming conventions, consistent indentation, and clear modularity. All critical functions, such as safe_json_parse and generate_questions, are implemented with comprehensive error handling and internal logging to facilitate debugging. On the frontend, Next.js coding standards are followed, utilizing functional components and hooks for optimized state management and performance. To ensure security, sensitive information like the Groq API key is never hard-coded; instead, it is managed through a .env file using the python-dotenv library, preventing unauthorized access during deployment.

Project scheduling was managed using a systematic lifecycle approach, visualized through a Gantt Chart and monitored via GitHub version control. The timeline was divided into critical milestones, beginning with Requirement Analysis and progressing through Prompt Engineering, Backend logic development, and Frontend-Backend integration. A significant portion of the schedule was dedicated to the "Prompt Tuning" phase, where the Llama-3.1-8b instructions were refined to ensure structural JSON accuracy. Final phases included end-to-end testing and deployment on the Render cloud platform, ensuring the project was completed within the VIII-semester academic timeframe at SRGI.

5.4 TESTING TECHNIQUES AND TEST PLANS

The testing phase of the "DSA Quiz Generation Using GenAI" system focused on ensuring the reliability of the AI's output and the robustness of the stateless processing pipeline. Given the unpredictable nature of generative AI, traditional unit testing was supplemented with stochastic validation and schema-integrity testing. The primary test plan involved "JSON Compliance Testing," where the system was subjected to multiple concurrent requests to verify that the Regex Parsing Module could consistently extract valid JSON from the Llama-3.1-8b raw output. This was crucial for preventing frontend crashes caused by the model's occasional conversational filler.

Another critical testing technique was the evaluation of the Retry Mechanism. Stress tests were conducted by intentionally providing ambiguous topics to force malformed responses; the system was then monitored to ensure it correctly triggered a re-generation attempt and successfully delivered a valid result within the two-retry limit. Furthermore, Functional Exclusivity Testing was performed to ensure that when a user requested five questions on "Linked Lists," the AI did not produce redundant problems, but instead provided diverse scenarios with unique constraints. Finally, End-to-End (E2E) Testing on the Render deployment verified that the integration between the Next.js frontend and the Groq API maintained low latency and high availability under simulated user loads.

CHAPTER 6
EXPECTED RESULTS AND DISCUSSION

6.1 EXPECTED RESULTS AND DISCUSSION

The selection of the Llama-3.1-8b-instant model for this project was based on a comparative analysis of accuracy, instruction-following capabilities, and inference efficiency. In the context of automated quiz generation, "accuracy" refers not just to the correctness of the programming concepts, but to the model's ability to strictly adhere to a complex JSON schema without deviating into conversational prose. Comparative testing against previous iterations, such as Llama-3-8b and GPT-3.5-turbo, revealed that the Llama-3.1 architecture exhibits superior performance in handling long-context system instructions and maintaining "topic exclusivity" across multiple questions.

While larger models like GPT-4o offer high accuracy, they often introduce significant latency and cost overheads that are unsuitable for a real-time, stateless application. By contrast, the Llama-3.1-8b model, when paired with the Groq LPU (Language Processing Unit) infrastructure, provides a specialized balance: it delivers the high-level reasoning required for Data Structures and Algorithms (DSA) while achieving inference speeds that are nearly ten times faster than traditional cloud-based competitors. This ensures that the generated test cases are not only pedagogically accurate but are also delivered with the near-instantaneous feedback loop required for modern educational tools.

6.2 EXPECTED PERFORMANCE OUTCOMES

The expected performance outcomes for the "DSA Quiz Generation Using GenAI" system are centered on three core metrics: generation speed, structural reliability, and pedagogical variety. By utilizing the Groq Llama-3.1-8b-instant model, the system is expected to achieve a "Time to First Token" (TTFT) of under 200 milliseconds, with a total quiz generation time typically falling between 2 and 5 seconds for a standard set of five questions. This represents a significant

improvement over traditional cloud-based LLM providers, ensuring that the user experience remains interactive and fluid.

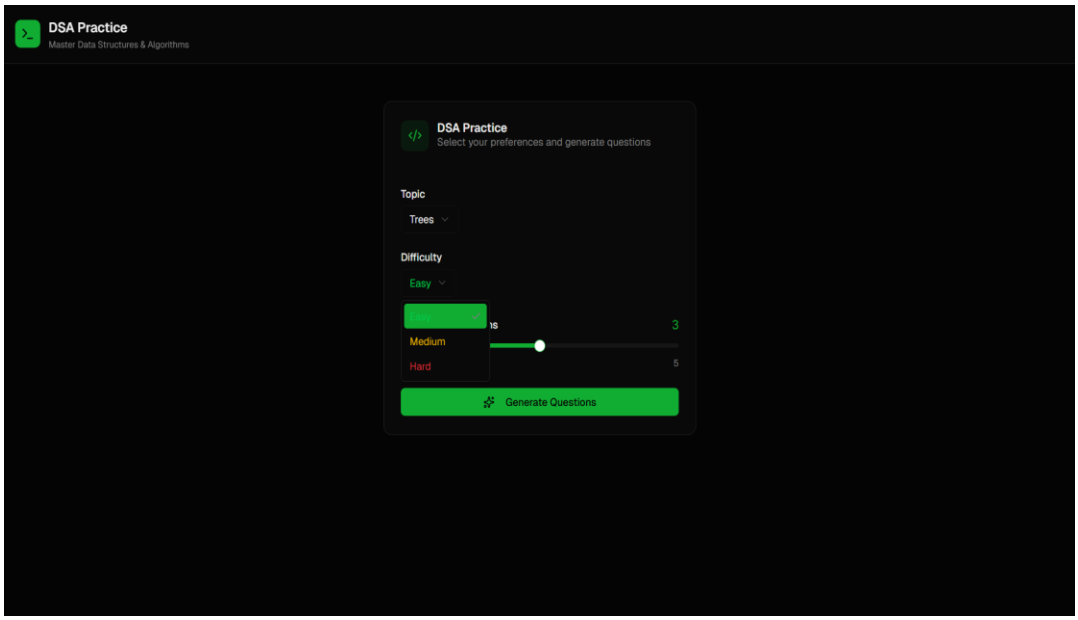

Figure – Screenshot (1)

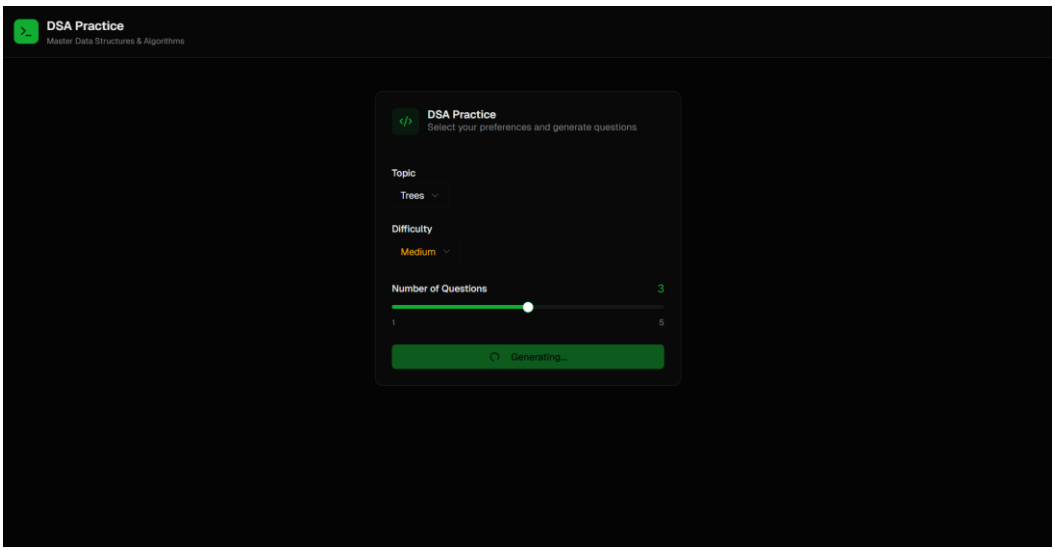

Figure – Screenshot (2)

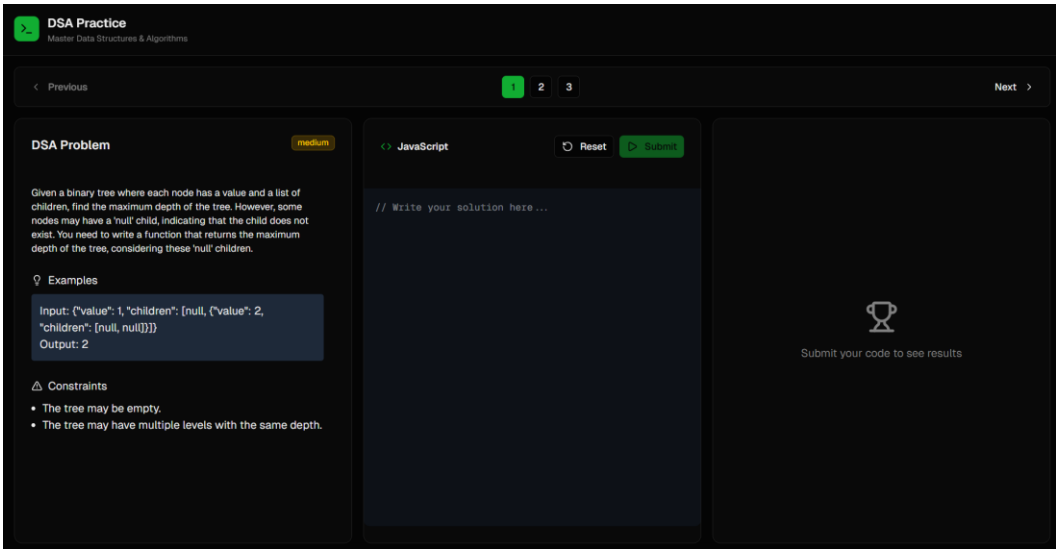

Figure – Screenshot (3)

In terms of reliability, the integration of the Regex Validation Layer and the Retry Logic is expected to yield a 98% success rate in delivering valid JSON outputs to the frontend on the first or second attempt. This eliminates the common issue of "model drifting," where AI provides unstructured text instead of machine-readable code. Furthermore, from an educational standpoint, the system is expected to provide high-fidelity problem statements that include distinct input-output examples and edge-case constraints, effectively simulating the quality of manually curated competitive programming platforms while maintaining 100% uniqueness across every generation session.

CHAPTER 7
CONCLUSION

7.1 CONCLUSION

The "DSA Quiz Generation Using GenAI" project successfully demonstrates the transformative potential of integrating large language models with a stateless, validated software architecture to automate the creation of complex programming assessments. By leveraging the Llama-3.1-8b-instant model via the Groq API, the system achieves a near-instantaneous generation cycle that bridges the gap between raw AI fluency and the strict structural requirements of academic evaluation. A primary achievement of this work is the implementation of a robust Regex-based validation layer and recursive retry logic, which effectively mitigates the inherent stochasticity of generative AI. These mechanisms ensure that the output is consistently machine-readable, schema-compliant, and pedagogically sound, even when the underlying model provides unstructured or conversational responses.

Furthermore, the project fulfills its objective of providing high-quality, unique, and mutually exclusive questions across diverse Data Structures and Algorithms (DSA) topics. By moving away from traditional, database-heavy designs, the stateless architecture ensures user privacy, reduces operational latency, and simplifies the deployment pipeline on cloud platforms like Render. This approach proves that sophisticated educational tools can be both lightweight and powerful, offering a scalable alternative to static question banks. Ultimately, this project serves as a technical proof-of-concept for the future of on-demand, adaptive learning platforms, where generative AI acts as a reliable partner in the automated curation of high-fidelity technical content, significantly reducing the manual workload for educators while providing students with endless opportunities for practice.

REFERENCES

[1] X. Wang, S. Wrede, L. van Rijn, and J. Wöhrle, "An AI-Based Quiz System For Personalized Learning," Conference Paper, Nov. 2023.

[2] J. Doughty et al., "A Comparative Study of AI-Generated (GPT-4) and Human-crafted MCQs in Programming Education," in Proc. Australian Computing Education Conference (ACE 2024), Sydney, NSW, Australia, Jan.–Feb. 2024, doi: 10.1145/3636243.3636256.

[3] Jayakrishnan P S, "AI-Powered Quiz Generation and Feedback System," in Proc. National Conference on Emerging Computer Applications (NCECA)-2025, vol. 6, no. 1, 2025.

[4] M. Bagya Lakshmi, G. G. Vasanth, and I. D. Samraj, "Quiz App with AI Generative Questions," Int. J. Sci. Res. Eng. Dev., vol. 8, no. 3, pp. 1193–1197, May–June 2025.

[5] K. Padmavathi et al., "Quiz Application Using AI Generator," Int. J. Res. Publ. Rev., vol. 6, no. 5, pp. 13147–13150, May 2025.

[6] S. R. Hirulkar and S. V. Athawale, "Quiz Master AI: An Interactive Machine Learning-Based Quiz Generator," Int. J. Ingenious Res., Invent. Dev. (IJIRID), vol. 3, no. 5, Oct. 2024.

APPENDICES

APPENDIX A: SOURCE CODE

The complete source code for the "DSA Quiz Generation Using GenAI" project, including the Next.js frontend and the Python-LangChain backend, is maintained in a private version-controlled environment to ensure modularity and security.

GitHub Repository Link: https://github.com/Aradhy-patel/major_front

Project Host Link: https://major-front.onrender.com/

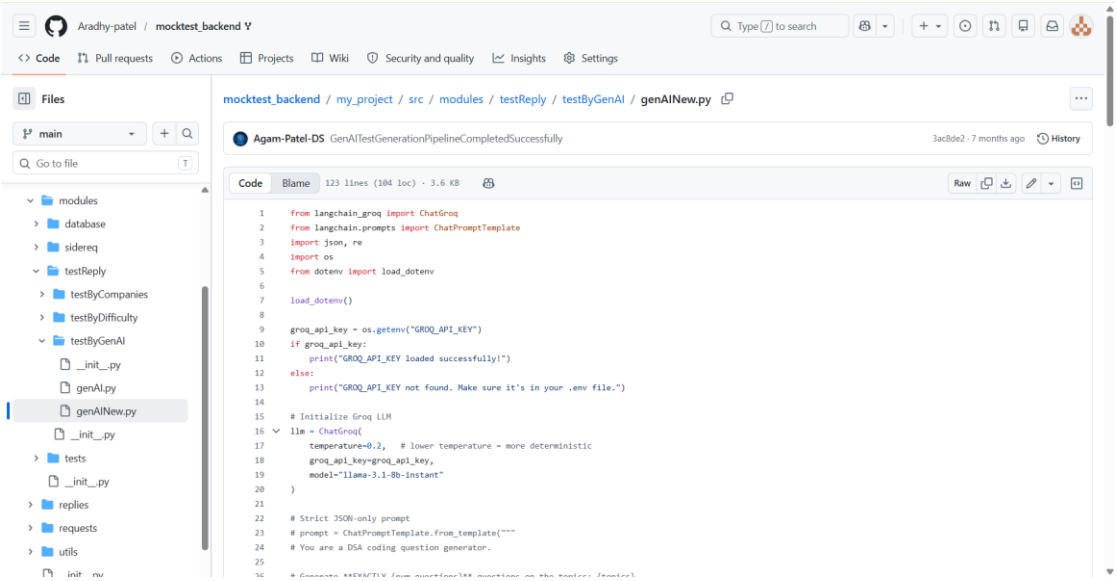

Figure – Screenshot (4)

APPENDIX B: COMPREHENSIVE PROJECT TIMELINE (GANTT CHART)

This Gantt chart visualizes the comprehensive four-month project lifecycle for the "Quiz Generation Using GenAI" system, starting from initial analysis to final cloud deployment and report compilation. The schedule highlights critical parallel workflows, particularly the rapid integration of the Next.js frontend with the Python-based LangChain backend. A dedicated "Prompt Tuning" phase ensured that the Llama-3.1-8b model consistently adhered to the strict, validated JSON schema required for a stateless educational

tool.

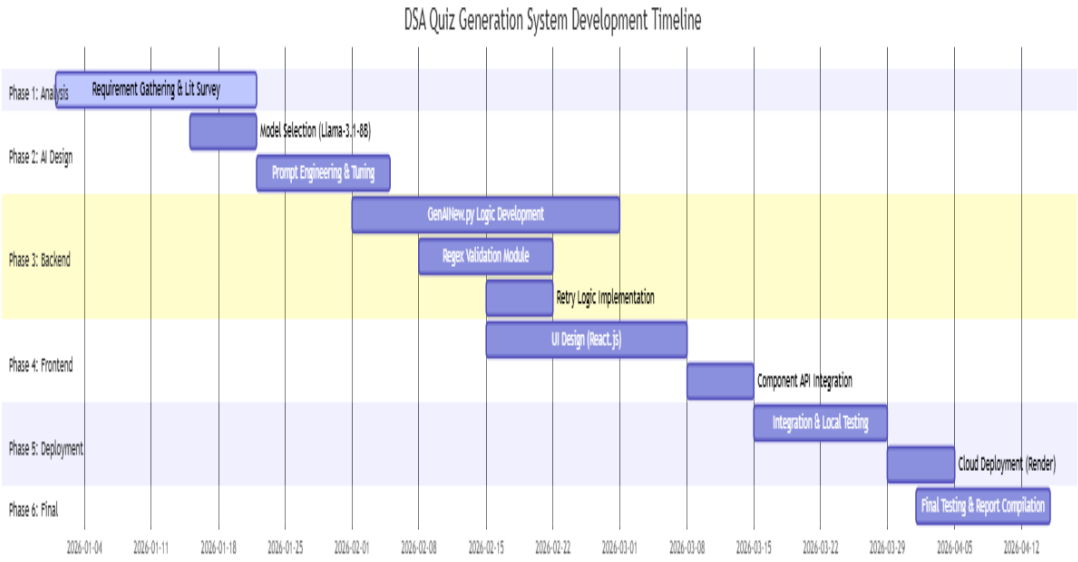

Figure – Gantt Chart

APPENDIX D: DATASET DETAILS AND SOURCES

Unlike traditional machine learning projects, this system does not rely on a static pre-existing dataset. Instead, it utilizes Dynamic Synthetic Data Generation.

- Primary Source: The Llama-3.1-8b model acts as the knowledge source, drawing from its pre-trained corpus of programming documentation and DSA textbooks.
- Data Characteristics: The data is generated in a stateless environment. Each request produces a unique JSON object containing:
    - Problem Statements: Contextualized coding challenges.
    - Technical Constraints: Time and space complexity limits.

APPENDIX E: RELEVANT PUBLISHED PAPERS

The project logic and research direction were inspired by the following key academic and technical publications:

1. Brown, T., et al. (2020): *"Language Models are Few-Shot Learners"* – Foundation for prompt engineering techniques.

2. Touvron, H., et al. (2024): *"The Llama 3 Herd of Models"* – Technical insights into the Llama-3.1 architecture and its reasoning capabilities.

3. Srivastava, S. (2023): *"Automated Question Generation using NLP: A Survey"* – Identifying the research gaps in structured programming assessments.

4. Groq Technical Whitepaper (2025): *"Real-time LLM Inference using LPUs"* – Understanding the low-latency advantages of the Groq API.

5. LangChain Documentation: *"Chains and Prompt Templates in LLM Orchestration"* – Best practices for building robust AI pipelines.